Academic References and Further Reading

EE5203 Microprocessors — where the full story is

The lectures in this course introduce each concept **conceptually**: enough mechanism to configure, use, and debug it, without the full architectural detail. This page maps every simplified concept to the authoritative document where the complete story lives, so that a curious student — or a future project — can go one level deeper with a known-good source instead of a random blog.

Each lecture gets its own section. Entries are added as lectures are revised.

If a link stops working

Links below point at the current official locations (verified 2026-07-17). If one rots:

- **ST documents** (RM…, PM…, UM…, MB… numbers): search the document number at st.com; the number uniquely identifies the document and survives site reorganizations.
- **Arm documents** (DDI… numbers): search at developer.arm.com.
- **Books**: available through the university library or the cited publisher; Lee & Seshia is free and legal at leeseshia.org.

Primary documents used throughout the course

Short name	Full citation	What it is for
RM0368	STMicroelectronics, *RM0368: STM32F401xB/C and STM32F401xD/E advanced Arm®-based 32-bit MCUs — Reference Manual* (PDF)	The register-level truth for every peripheral on our chip: GPIO, SYSCFG, EXTI, timers, ADC, I2C, SPI, DMA

Short name	Full citation	What it is for
PM0214	STMicroelectronics, *PM0214: STM32 Cortex®-M4 MCUs and MPUs programming manual* (PDF)	The processor core seen from software: instruction set, exception model, NVIC, SysTick, core registers
UM1724	STMicroelectronics, *UM1724: STM32 Nucleo-64 boards (MB1136) — User Manual* (PDF)	The board itself: B1 and LD2 circuits, jumpers, ST-LINK, power, **board revision differences**
MB1136 schematic	STMicroelectronics, *MB1136 schematic pack*, rev C-04 (PDF)	The actual circuit diagram of the Nucleo-F401RE — the version used in lecture
UM1725	STMicroelectronics, *UM1725: Description of STM32F4 HAL and low-layer drivers* (PDF)	Signature and behavior of every `HAL_…` function the labs use
Yiu 2014	J. Yiu, *The Definitive Guide to ARM® Cortex®-M3 and Cortex®-M4 Processors*, 3rd ed., Newnes, 2014. ISBN 978-0-12-408082-9	The standard book-length treatment of the core: pipeline, exceptions, NVIC, low-power, debug
DDI 0403	Arm Ltd., *ARMv7-M Architecture Reference Manual* (Arm DDI 0403)	The formal architecture specification — for when even Yiu is not precise enough
Lee & Seshia	E. A. Lee and S. A. Seshia, *Introduction to Embedded Systems: A Cyber-Physical Systems Approach*, 2nd ed., MIT Press, 2017 — free PDF	The academic textbook view: interrupts, concurrency, real-time reasoning
White 2024	E. White, *Making Embedded Systems*, 2nd ed., O'Reilly, 2024	Practical engineering judgment: interrupt hygiene, debouncing, system design patterns
C11	ISO/IEC 9899:2011, *Programming languages — C* (link is N1570, the final public draft, PDF)	The C language standard; final authority on `volatile`, integer types, undefined behavior

L01 — Embedded Systems, the C Toolchain, and Debugging

Concept as taught	Where the full story is
What an embedded system is — dedicated function + constraints	Lee & Seshia, ch. 1 *Introduction* (free PDF); White 2024, ch. 1
The Nucleo-F401RE board — ST-LINK, LD2, B1, headers	UM1724 Rev 17, §6 *Hardware layout* — p. 13; MB1136 C-04 schematic
Processor / MCU / system model	PM0214 Rev 10, §2.1 *Programmers model* — p. 17; Yiu 2014, ch. 1–2
The toolchain — compile, link, flash, debug	Yiu 2014, ch. 2 (development flow and what CMSIS standardizes); the build console of any CubeIDE project shows each stage's real output
First C statements — `main()`, declarations, `if`, `while (1)`	C11 (N1570), §6.8.4 *Selection statements* — p. 166; program startup is §5.1.2.2.1

L02 — Values, Decisions, Loops, and Arrays

Concept as taught	Where the full story is
`if` / `else if` / `else`	C11 (N1570), §6.8.4 *Selection statements* — p. 166
`while` and `for` loops	C11 (N1570), §6.8.5 *Iteration statements* — p. 168
Arrays and indexing from zero	C11 (N1570), §6.5.2.1 *Array subscripting* — p. 98 — out-of-range access is undefined behavior, which is why `samples[8]` is not "just the next value"
`uint16_t` and friends	C11 (N1570), §7.20 *Integer types* `<stdint.h>` — p. 307
Integer division — `a / b` truncates	C11 §6.5.5 (multiplicative operators): division truncates toward zero

L03 — Functions, Arrays, Strings, and Multiple C Files

Concept as taught	Where the full story is
Declarations, definitions, calls	C11 (N1570), §6.9.1 *Function definitions* — p. 174
Pass by value — parameters are copies	C11 (N1570), §6.5.2.2 *Function calls* — p. 99

Concept as taught	Where the full story is
Headers, multiple files, the linker	C11 (N1570), §6.2.2 *Linkages of identifiers* — p. 54; `#include` / `#define` are §6.10 *Preprocessing directives* — p. 184
Why array parameters need a count — arrays decay to pointers	C11 §6.7.6.3¶7; the L04 pointer material makes this concrete
C strings and `\0`; `strcmp`	C11 (N1570), §7.24 *String handling* `<string.h>` — p. 380

L04 — Pointers, Structures, Bits, Registers, and GPIO

The lecture builds `GPIOA->BSRR = (1U << 5);` from its parts: addresses, pointers, structures, masks, and the GPIO registers.

Concept as taught	Where the full story is
Addresses and memory — "memory is a numbered collection of storage locations"	PM0214 Rev 10, §2.2 *Memory model* — p. 28: the real Cortex-M4 memory map, including where peripherals live
Pointers: `&` and `*`	C11 (N1570), §6.5.3.2 *Address and indirection operators* — p. 106
Structures and `.` / `->`	C11 (N1570), §6.7.2.1 *Structure and union specifiers* — p. 130
Bitwise operators, masks, the `U` suffix	C11 (N1570), §6.5.7 *Bitwise shift operators* — p. 112, then §6.5.10–6.5.12 for `&`, `^`, `|`; §6.5.7 also explains why shifting **signed** values can be undefined behavior — the reason the `U` suffix matters
CMSIS register names — "the device header supplies the definition"	The actual `stm32f401xe.h` device header (`GPIO_TypeDef`, `GPIOA_BASE`); Yiu 2014, ch. 2 on what CMSIS standardizes
`volatile` register members	C11 (N1570), §6.7.3 — p. 140; M. Barr, *How to Use C's volatile Keyword*
Peripheral clock gating — "enable the clock first"	RM0368 Rev 6, `RCC_AHB1ENR` — p. 118; the RCC chapter (§6) explains *why* clocks are gated: power
GPIO block, `MODER`, `BSRR`	RM0368 Rev 6, §8 *GPIO* — p. 146; registers: `GPIOx_MODER` — p. 158, `GPIOx_BSRR` — p. 161 — including what the lecture skips: OTYPER, OSPEEDR, PUPDR

L05 — GPIO and Hardware Abstraction

The lecture connects every HAL name to the register behavior taught in L04: same hardware, second interface.

Concept as taught	Where the full story is
The HAL GPIO API — `GPIO_InitTypeDef`, `HAL_GPIO_Init`, `WritePin` / `TogglePin` / `ReadPin`	UM1725 Rev 8, GPIO functions — p. 447
What the HAL actually executes	The source of `stm32f4xx_hal_gpio.c` — `HAL_GPIO_WritePin` is a single `BSRR` write
HAL names → register bits — `GPIO_MODE_OUTPUT_PP`, `GPIO_NOPULL`, speed	RM0368 Rev 6, §8 *GPIO* — p. 146; registers: `GPIOx_MODER` — p. 158, `GPIOx_PUPDR` — p. 159, `GPIOx_BSRR` — p. 161
Floating inputs, pull-up / pull-down	RM0368 §8.3 (I/O port control); the electronics context in any circuits text — the lecture's claim is only that an undriven CMOS input is undefined
B1 circuit — external pull-up, active-low	MB1136 schematic rev C-04; UM1724 Rev 17, push-buttons — p. 24; confirmed against the ST Nucleo BSP source
Why abstraction layers exist — and what they cost	White 2024, ch. 2 (creating system diagrams / layering); Yiu 2014, ch. 2 (what CMSIS standardizes and why)
Polling and its limits — "the loop asks again and again"	Forward pointer: the L06 row *Polling vs. interrupts* above

L06 — GPIO Interrupts

The lecture walks one button press through five plain-language stages (PRESS → NOTICE → ASK → LOOK UP → RUN YOUR CODE). Each simplification below names the document that removes it.

Page-anchored links (`#page=N`) below open the PDF at the cited page in the browser viewer. Page numbers were verified against the document revisions named in each entry (2026-07-17); if ST publishes a new revision, the section numbers in the text remain the reliable pointer.

Concept as taught	Where the full story is
Polling vs. interrupts — "polling asks again and again"	Lee & Seshia, ch. 10 *Input and Output* (interrupt mechanisms, latency, and when polling is actually the better design); Yiu 2014, §4.5
What really happens between ASK and RUN YOUR CODE — register stacking, vector fetch, exception entry/return (we skipped this entirely)	PM0214 Rev 10, §2.3 *Exception model* — p. 37; Yiu 2014, ch. 8 (including the 12-cycle entry sequence and tail-chaining); DDI 0403, §B1.5 for the formal definition
EXTI — "a set of hardware edge detectors"	RM0368 Rev 6, §10 *EXTI* — p. 205: block diagram, `IMR/EMR/RTSR/FTSR/SWIER/PR` registers, the event-vs-interrupt distinction we did not cover
SYSCFG routing — "a four-bit field selects the port"	RM0368 Rev 6, `SYSCFG_EXTICR4` — p. 143
Pending bit and write-1-to-clear	RM0368 Rev 6, `EXTI_PR` — p. 211; the same rc_w1 convention appears in many STM32 peripherals
NVIC — "enable, priority, deliver"	PM0214 Rev 10, §4.2 *NVIC* — p. 208; Yiu 2014, ch. 7 — including what we postponed: priority levels, preemption, sub-priority, nesting
Vector table and the startup file	PM0214 Rev 10, §2.3.4 *Vector table* — p. 40; Yiu 2014, §4.4 and §8.5; the generated `startup_stm32f401retx.s` in every CubeIDE project is the concrete instance
`EXTI15_10_IRQHandler` / HAL helper / callback chain	UM1725 Rev 8, GPIO functions — p. 447 (`HAL_GPIO_EXTI_IRQHandler`, `HAL_GPIO_EXTI_Callback`); the ~10-line source of the helper in `stm32f4xx_hal_gpio.c` is worth reading in the lab
`volatile` — "read the stored value each time"	C11 (N1570), §6.7.3 — p. 140; M. Barr, *How to Use C's volatile Keyword*, Barr Group. Note what the lecture only hints at: `volatile` does **not** make read-modify-write atomic — see Lee & Seshia, ch. 11 on shared-variable concurrency

Concept as taught	Where the full story is
ISR main() shared-variable pattern (`button_event`)	Lee & Seshia, ch. 11 (race conditions, atomicity); White 2024, interrupt chapter (keep-ISRs-short discipline and its reasons)
Switch bounce and the 80 ms guard	J. Ganssle, *A Guide to Debouncing*, 2nd ed., The Ganssle Group, 2008: measured bounce data for real switches (the "settles within ~10 ms" claim comes from here), plus hardware and software debounce alternatives
B1 circuit — external pull-up, active-low	MB1136 schematic rev C-04; UM1724 Rev 17, push-buttons — p. 24 — **check the board revision**: later MB1136 revisions changed the B1 circuit, which affects the pull configuration the lab assumes

L07 — Timers

The lecture builds one mental model — clock → `PSC` → `CNT` → `ARR` → update event — and consumes it by polling, by interrupt, and as a 1 ms tick.

Concept as taught	Where the full story is
The time base — prescaler, counter, auto-reload, update event	RM0368 Rev 6, §13 *TIM2 to TIM5* — p. 316; registers (§13.4 — p. 353)
Where the 16 MHz comes from — clock tree, APB timer-clock ×2 rule	RM0368 Rev 6, §6.2 *Clocks* — p. 94
HAL TIM base API — handle, `Base_Init`, `Base_Start_IT`, callback	UM1725 Rev 8, TIM functions — p. 1065
The interrupt path — reused from L06	The L06 rows above; the only new element is the request name `TIM2_IRQn`
Tick-based scheduling and why blocking delays hurt	Lee & Seshia, ch. 10–11; White 2024, timer and main-loop chapters
SysTick — the Cortex-M core timer behind `HAL_Delay` (not covered)	PM0214 Rev 10, §4.5 *SysTick timer* — p. 246

L08 — Pulse-Width Modulation

The lecture adds the capture/compare channel to L07's time base and uses it in the compare direction: `ARR` owns the frequency, `CCR` owns the duty.

Concept as taught	Where the full story is
PWM mode — pin high while `CNT < CCR`	RM0368 Rev 6, §13.3.9 *PWM mode* — p. 336, including what the lecture skips: center-aligned mode, PWM mode 2
Channel registers	`TIMx_CCMR1` — p. 362, `TIMx_CCER` — p. 366, `TIMx_CCR1` — p. 367
Handing the pin to the timer — alternate function	RM0368 Rev 6, `GPIOx_AFRL` — p. 162; the pin-to-AF table is the STM32F401RE datasheet (DS10086), *Alternate function mapping* — search the DS number at st.com
HAL PWM API	UM1725 Rev 8, TIM functions — p. 1065
The servo pulse contract — 50 Hz frame, 1–2 ms pulse	Hobby-servo convention, not an ST document: any RC-servo datasheet (e.g., SG90, S3003) states the same 20 ms / 1–2 ms numbers
Why the average works — PWM into slow loads	White 2024, output/actuator chapters; the RC-filter demo linked from the deck shows it interactively

L09 — Input Capture

The same channel, reversed: an edge on the pin captures the free-running counter into `CCR` — a hardware timestamp.

Concept as taught	Where the full story is
Input capture — edge copies `CNT` into `CCR`	RM0368 Rev 6, §13.3.5 *Input capture mode* — p. 333
Automated duty measurement	RM0368 Rev 6, §13.3.6 *PWM input mode* — p. 334 — the hardware version of the paired-channel trick, at the cost of the whole timer
Direct vs indirect inputs, the filter, overcapture	`TIMx_CCMR1` — p. 362 (`CCxS`, `ICF`), `TIMx_CCER` — p. 366; overcapture flags in the `TIMx_SR` description
HAL capture API	UM1725 Rev 8, TIM functions — p. 1065; `HAL_TIM_ReadCapturedValue` — p. 1097

Concept as taught	Where the full story is
Ultrasonic ranging — echo width → distance	HC-SR04 datasheet (community-published; no ST equivalent): 10 µs trigger, echo width, the /58 cm constant

L10 — Analog-to-Digital Conversion

The lecture builds counts volts fluency on one convention (`mV = raw × 3300 / 4095`), one clock (PCLK2 ÷ 2 = 8 MHz), and one converter (the SAR core).

Concept as taught	Where the full story is
The ADC itself — 12-bit SAR, one converter on the F401	RM0368 Rev 6, §11 *ADC* — p. 213
The ADC clock — PCLK2 ÷ 2 = 8 MHz, 36 MHz ceiling	§11.3.2 *ADC clock* — p. 215
Channels — 16 external doors + the internal sensors	§11.3.3 *Channel selection* — p. 215; §11.9 *Temperature sensor* — p. 226
Single vs continuous conversion	§11.3.4–11.3.5 — p. 216; what the lecture skips: injected channels, the analog watchdog, external triggers (§11.6 — p. 222)
Sampling time and source impedance	§11.5 *Channel-wise programmable sampling time* — p. 222; the hard electrical limits (max source resistance vs sampling time) are in the STM32F401RE datasheet (DS10086), *ADC characteristics*
ADC registers	§11.12 — p. 229: `ADC_SR` — p. 229, `ADC_CR1` — p. 230, `ADC_CR2` — p. 232, `ADC_SMPR2` — p. 234, `ADC_SQR3` — p. 237, `ADC_DR` — p. 239
The analog pin mode — `MODER = 11`	RM0368 Rev 6, `GPIOx_MODER` — p. 158
HAL ADC API	UM1725 Rev 8, ADC generic driver — p. 56: `HAL_ADC_Start` — p. 65, `HAL_ADC_PollForConversion` / `HAL_ADC_Start_IT` — p. 66, `HAL_ADC_GetValue` / `HAL_ADC_ConvCpltCallback` — p. 68, `HAL_ADC_ConfigChannel` — p. 69
Sampling theory and aliasing — stated, not proven	Lee & Seshia, sensors/actuators and sampling chapters; White 2024, analog-input chapters

L11 — The Bridge to Assembly

Every listing in the deck is real output of `arm-none-eabi-gcc` 14.3 — the toolchain inside STM32CubeIDE — compiled for Cortex-M4 at `-O0` and `-O2`.

Concept as taught	Where the full story is
The register file — `R0`–`R12`, `SP`, `LR`, `PC`, `xPSR`	PM0214 Rev 10, §2.1.3 *Core registers* — p. 18
Thumb-2 — 16- and 32-bit encodings, the `.w` suffix	PM0214 Rev 10, §3.1 *Instruction set summary* — p. 50; §3.3.8 *Instruction width selection* — p. 68
Memory access — `LDR`/`STR`, offsets, `PUSH`/`POP`	§3.4 — p. 69; §3.4.2 *immediate offset* — p. 71; §3.4.7 *PUSH and POP* — p. 78
Data processing — arithmetic, logic, shifts, `CMP`, `MOV`	§3.5 — p. 81: `ADD/SUB` — p. 83, `AND/ORR/EOR/BIC` — p. 85, shifts — p. 62, `CMP` — p. 88, `MOV/MVN` — p. 89
Branches and conditions — `B`/`BL`/`BX`, condition codes, `IT`	*B, BL, BX, BLX* — p. 142; §3.3.7 *Conditional execution* — p. 65; the `-O2` `ITE` surprise: *IT* — p. 145
Calling convention — args in `r0`–`r3`, result in `r0`, `LR`	The Arm AAPCS32 (*Procedure Call Standard for the Arm Architecture*) — search "AAPCS32" at developer.arm.com; the lecture teaches only the subset visible in the debugger
Interrupt entry stacking — same stack, hardware-driven	the L06 rows above (exception model); the deck only points back to them
What the optimizer may do — folding, inlining, `volatile` limits	GCC manual, *Optimize Options*; White 2024, debugging/optimization chapters

Sections for other lectures are added as each lecture is revised to the 2026–2027 pattern.